Лабораторна робота №2

Функціональне програмування

Завдання 3. Паралельний метод Гаусса

Левківський Євгеній Сергійович

ІПС-41

1. Мета роботи

Реалізувати багатопотоковий застосунок на Haskell для розв’язання систем лінійних рівнянь паралельним методом Гаусса. Використати GHC з ключем -threaded, перевірити правильність і дослідити час виконання.

2. Постановка завдання

Обрано третє завдання: «Вирішити систему лінійних рівнянь паралельним варіантом методу Гаусса». Вхідні дані — квадратна матриця A і вектор правих частин b. Необхідно знайти вектор x, що задовольняє Ax=b, або повідомити, що єдине чисельно стійке рішення не визначається.

Gaussian.hs містить чисельний алгоритм, WorkerPool.hs — обмежений пул потоків, Main.hs — читання файла, команди й вимірювання. Вхідний формат: N, потім N рядків по N коефіцієнтів та одному значенню правої частини. Допускаються порожні рядки й коментарі після #.

3. Послідовність обчислень

Спочатку перевіряються розміри, квадратність матриці та скінченність чисел. Кожне рівняння ділиться на найбільший за модулем коефіцієнт його рядка. Це зменшує вплив різних масштабів рівнянь. Рядок із нульовими коефіцієнтами означає, що єдиного рішення немає.

На кроці k обирається рядок із найбільшим за модулем коефіцієнтом у стовпці k серед ще не опрацьованих рядків. За потреби рядки переставляються. Якщо модуль головного елемента після нормалізації не перевищує 1e-12, повертається SingularSystem.

Для кожного нижнього рядка i обчислюється множник m=aik/akk та виконується перетворення a'ij=aij-m·akj; права частина змінюється так само. Після завершення прямого ходу матриця стає верхньою трикутною. Зворотний хід знаходить невідомі від останнього рядка до першого.

Формула зворотного ходу: xi=(bi−Σ aij·xj)/aii, де сума береться за j>i. Числа зберігаються як Double. Переповнення або нескінченний проміжний результат дає NumericalFailure. Максимальний порядок у програмі — 2000.

4. Організація паралельності

На одному кроці перетворення різних нижніх рядків незалежні. Головний рядок є спільним незмінним вхідним значенням. Завдання передаються фіксованому пулу через Chan; forkIO запускає працівників. Результати мають індекси й повертаються в початковому порядку.

Після кожної фази виконується бар'єр: вибір наступного головного елемента починається лише після всіх оновлень. Кількість працівників не перевищує задану. Пул створюється для кожної фази; це проста схема, але витрати створення потоків помітні на невеликих матрицях.

UArray зберігає обчислені числа в щільному масиві. Evaluate та force забезпечують завершення результату в працівнику. Синхронні помилки передаються головному потоку; bracket прибирає працівників. Асинхронне завершення не поглинається обробником помилок.

Збирання використовує -O2 -Wall -threaded -rtsopts. Параметр +RTS -N4 дозволяє виконувати Haskell-код на чотирьох ядрах. Самі forkIO без декількох capabilities не гарантують багатоядерного виконання.

5. Приклад і перевірка рішення

```
3
2 1 -1 8
-3 -1 2 -11
-2 1 2 -3

lab2 solve 4 lab2/examples/system.txt +RTS -N4 -RTS
```

Отримано x1=2, x2=3, x3=−1. Підстановка дає 8, −11 та −3 відповідно до правих частин. Максимальна абсолютна нев'язка дорівнює 0; відносна також 0. Фактичний вивід збережено в lab2/results.txt.

Абсолютна нев'язка визначається як ||Ax−b||∞. Відносна: ||Ax−b||∞/(||A||∞·||x||∞+||b||∞), де норма матриці — максимальна сума модулів у рядку. Додатковий приклад pivot.txt має нульовий перший діагональний елемент і перевіряє перестановку рядків; результат x=(1,2).

Автоматичні тести охоплюють системи з відомим рішенням порядків 1, 2, 3, 5, 8, 16, 32, різні масштаби рівнянь, перестановки, вироджені й

несумісні системи, неправильні розміри та нескінченні значення. Послідовний і паралельний результати для тієї самої матриці збігаються.

6. Експеримент зі швидкодією

Використано Windows x64, Ryzen 5 5600H, 12 логічних процесорів, GHC 9.6.7. Генерується діагонально переважна матриця порядку 384 з відомим x; b обчислюється як Ax. Вхідні дані створено до таймера. Один прогрівальний запуск виключено.

Для 1, 2 та 4 працівників виконано по три запуски, використано медіану. Перед запуском виконується performGC, час вимірюється монотонним getMonotonicTimeNSec. Таймер охоплює нормалізацію, прямий і зворотний хід та організацію потоків. Після таймера перевіряються відносна нев'язка <1e-10 та похибка відомого рішення <1e-8.

Працівники	**Медіана, с**	**Прискорення**	**Ефективність**
1	0.340895	1.0000	1.0000
2	0.273305	1.2473	0.6237
4	0.362693	0.9399	0.2350

S(k)=T(1)/T(k), E(k)=S(k)/k. На цій матриці два працівники дали прискорення близько 1,25. Чотири виявилися повільнішими через витрати потоків, бар'єри й доступ до пам'яті. Паралельність не гарантує прискорення для кожного розміру. Дані не замінено теоретичними оцінками.

Повторення: node scripts/benchmark.mjs. CSV збережено в lab2/benchmark.csv, журнал — verification/benchmark.txt, параметри системи — verification/environment.json.

7. Висновок

Виконано третє завдання: реалізовано паралельний метод Гаусса з нормалізацією та частковим вибором головного елемента. Підтверджено правильність нев'язкою, відомими рішеннями та порівнянням режимів. Обчислювальна складність — $O(n^3)$, пам'ять — $O(n^2)$. Реалізація використовує Double й повідомляє про чисельно вироджені системи замість

некоректного ділення.

8. Джерела

GHC User’s Guide, parallel runtime: https://downloads.haskell.org/ghc/9.6.7/docs/users_guide/

Бібліотеки Haskell base, array, deepseq: https://hackage.haskell.org/